



Formation Kubernetes

Session 10 — Production Readiness

Résilience, autoscaling, monitoring et sécurité

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session



Health Probes

Liveness, Readiness, Startup : garder les pods sains

15 min



Resource Management

Requests vs Limits, QoS, scheduling

15 min



Horizontal Pod Autoscaler (HPA)

Scaling automatique basé sur les métriques

15 min



VPA + Cluster Autoscaler

Ajustement ressources et scaling nodes

15 min



Résilience & Sécurité

PDB, NetworkPolicy, RBAC, Pod Security Standards

20 min

Recap — Session 9



Terraform Providers

kubernetes + helm : déployer K8s comme du code



Module application/

Abstraction : Deployment, Service, Ingress en un bloc



Makefile DX

make init, make plan, make apply — workflow simple

Question : on a du code Terraform. Comment s'assurer qu'il tient la prod ?

Health Probes – Les trois piliers

Liveness Probe

Est-il VIVANT ?

Détecte si le processus est mort/bloqué

Action :

Redémarrer le pod

/health

Readiness Probe

Peut-il servir ?

Détecte si le pod peut recevoir du trafic

Action :

Retirer du service LB

/ready

Startup Probe

Est-il PRÊT ?

Donne du temps au démarrage

Action :

Attendre avant liveness

/startup

Erreur courante : faire un liveness probe qui vérifie la DB. Si la DB est down, le pod restart en boucle.

Health Probes – Configuration YAML

```
containers:
- name: api
  image: api:v1
  livenessProbe:
    httpGet:
      path: /health
      port: 8080
    initialDelaySeconds: 10
    periodSeconds: 10
    failureThreshold: 3
  readinessProbe:
    httpGet:
      path: /ready
      port: 8080
    initialDelaySeconds: 5
    periodSeconds: 5
```

livenessProbe

Redémarrer si faux

initialDelay

Attendre avant 1er check

readinessProbe

Retirer du LB si faux

Resource Management – Requests vs Limits

Requests

Garantie : K8s réserve pour l'app

Utilisée

Limits

Limite : K8s tue si on dépasse

Maximale

Classes de Qualité de Service (QoS)

Guaranteed

`requests = limits`

Les plus prioritaires

Burstable

`requests < limits`

Peuvent être évictées

HPA – Horizontal Pod Autoscaler

Flux de décision du HPA



```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    kind: Deployment
    name: api
  minReplicas: 2
  maxReplicas: 5
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

Paramètres clés :

- 1 HPA par Deployment
- minReplicas/maxReplicas
- Métriques : CPU, RAM, custom
- Scaling cooldown : 3-5 min
- ⚠ Requiert metrics-server

VPA & Cluster Autoscaler – 3-tier scaling

Tier 1: HPA



Horizontal Pod Autoscaler

Ajouter des PODS

Tier 2: VPA



Vertical Pod Autoscaler

Ajuster REQUESTS/LIMITS

Tier 3: CA



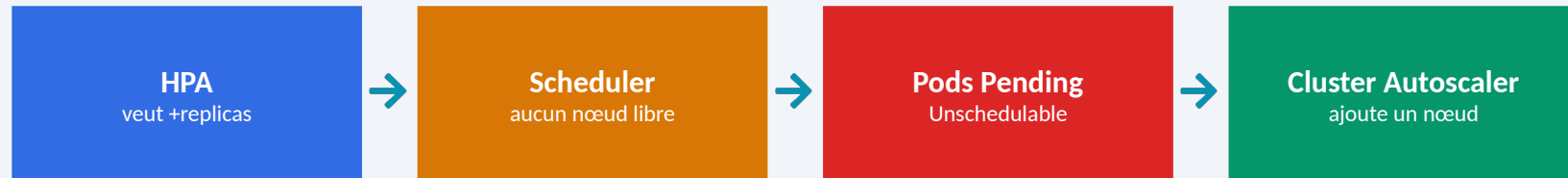
Cluster Autoscaler

Ajouter des NODES

Quand ça monte : HPA d'abord → si pods can't fit, Cluster Autoscaler ajoute des nodes

Scaling – quand le HPA ne suffit pas

Le HPA crée des pods... encore faut-il des nœuds pour les placer



Si le Cluster Autoscaler est au max (quota régional, max-nodes, pas de capacité) → les pods restent Pending : pas de scale réel.

Cluster Autoscaler	Karpenter	Spot / Preemptible
• Built-in sur GKE • Ajoute des nœuds quand des pods sont Pending • Borné par max-nodes + quota	• Provisioning just-in-time • Choisit le type d'instance, • bin-packing + consolidation • Plus rapide, spot natif	• -60 à -90 % de coût • Gérer l'interruption : • PDB + arrêt propre • (cluster training = e2 spot)

PodDisruptionBudget – Zéro downtime

Événements qui évicent les pods

Node Drain

Maintenance hardware, K8s upgrade

Cluster Upgrade

GKE control plane patch

Preemption

Node récupéré pour une workload priority

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
spec:
  minAvailable: 1
  # OR:
  # maxUnavailable: 1
  selector:
    matchLabels:
      app: api
```

minAvailable

Garder au moins N pods up

maxUnavailable

Permettre au max N pods down

Sélecteur

Quels pods protéger (labels)

Exemple : 3 replicas API, PDB minAvailable: 1 → pendant un drain, K8s garde toujours ≥1 pod up

NetworkPolicy – Segmentation réseau



Défaut

Tous les pods
peuvent talk

OUVERT



Avec NetworkPolicy

Deny all sauf
exceptions

FERMÉ

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-from-frontend
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
    - Ingress
  ingress:
    - from:
        podSelector:
          matchLabels:
            app: frontend
      ports:
        - protocol: TCP
          port: 8080
```

Observabilité — Les 3 piliers

Logs



- Qu'est-ce qui s'est passé?
- Error traces, stack dumps

ELK / Loki

Metrics



- Ça va bien?
- CPU, RAM, latency, errors

Prometheus + Grafana

Traces

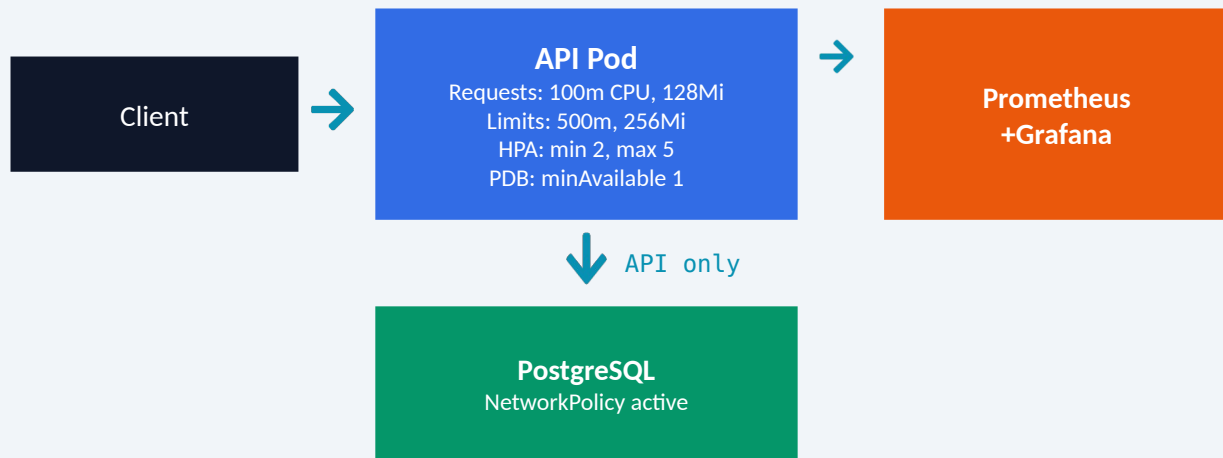


- Ça prend combien de temps?
- Path d'une requête

Jaeger / Otel

Sans monitoring, vous déployez à l'aveugle. Avec les 3 piliers, vous voyez tout.

Architecture du TP – Toutes les briques



Étapes du TP (1h30)

1. Health Probes

2. Resource requests/limits

3. HPA

4. PDB

5. NetworkPolicy

6. Prometheus + Grafana

Bonus: Load test avec `hey` → voir HPA scale

Le module application/ — le code réel (cloud-infrastructure)

```
# cloud-infrastructure/modules/application/main.tf
resources {
  requests = { cpu = var.resource_requests_cpu, memory = var.resource_requests_memory }
  limits   = { cpu = var.resource_limits_cpu,    memory = var.resource_limits_memory }
}

liveness_probe {
  http_get { path = var.health_check_path, port = local.default_port }
  initial_delay_seconds = var.health_check_initial_delay_seconds
  period_seconds        = var.health_check_period_seconds
  failure_threshold     = var.health_check_failure_threshold
}

readiness_probe {
  http_get { path = var.readiness_check_path, port = local.default_port }
}

dynamic "startup_probe" { for_each = var.startup_probe_enabled ? [1] : [] ... }
```

Toutes les features prod du TP (probes, requests/limits) — encapsulées une seule fois dans le module Terraform

Do's & Don'ts – Production Readiness

À FAIRE

- requests = plancher réaliste (scheduling + base du HPA)
- limits pour limiter le blast radius (OOM/throttle localisé)
- readiness = dépend des dépendances (DB, cache)
- liveness = juste « le process répond »
- 1 PDB par workload critique
- tester le scaling sous charge réelle avant la prod

À ÉVITER

- pas de requests → HPA aveugle + évincé en premier (BestEffort)
- liveness qui tape la DB ou /ready → boucle de redémarrage
- limits == requests trop serrés → OOMKilled / throttling
- HPA + VPA sur la même métrique (ils se battent)
- NetworkPolicy sans dataplane qui l'applique → ouverte en silence
- drain / upgrade sans PDB → downtime

Récapitulatif & Mini-défi

4 clés pour la Production Readiness

1

Santé : Health Probes détectent les pods malades

2

Ressources : Requests pour le scheduling, Limits pour la protection

3

Scaling : HPA pour la charge, Cluster CA pour les nodes

4

Résilience : PDB, NetworkPolicy, RBAC, monitoring

Mini-défi pour les plus avancés

Ajouter VPA en 'Recommendation Mode' → consulter les recommandations et valider